

Vector Search, Semantic Search & Hybrid Search

Day 4 — 4-Day Elasticsearch Course

Elasticsearch 8.19 | dense_vector · ELSER · semantic_text · RRF

Agenda

1. Vector Search Fundamentals
2. **Practice 4A** — Vector Search
3. ELSER & Semantic Text
4. **Practice 4B** — Semantic Search
5. Hybrid Search with RRF
6. **Practice 4C** — Hybrid Search
7. Advanced Topics
8. **Practice 4D** — Advanced (Bonus)
9. Course Wrap-up

Stack Check

Today requires the **elk-m1** stack (8GB+ RAM):

```
# Should be running already
docker compose -f docker/elk-m1/docker-compose.yml --env-file .env ps

# If not, start it
docker compose -f docker/elk-m1/docker-compose.yml --env-file .env up -d
```

Endpoints:

- Elasticsearch: `https://localhost:9200` (auth: `elastic` / `elastic`)
- Kibana: `http://localhost:5601`

HTTPS for this stack — Kibana handles the certificate internally.

Vector Search Fundamentals

From keywords to meaning

The Evolution of Search

Keyword Search (BM25)

"space adventure"



Exact term matching

"space" AND "adventure"

Vector Search

[0.2, 0.8, ...]



Semantic similarity

"cosmic journey" ≈ match

Hybrid Search

BM25 + kNN + RRF



Best of both worlds

Exact + semantic

Generation	Technology	Weakness
Keyword	BM25 (inverted index)	Misses synonyms, paraphrases
Semantic	Embeddings + kNN	May miss exact terms, proper nouns
Hybrid	BM25 + kNN + RRF	More complex, but most accurate

What Are Embeddings?

Embeddings are **numerical representations** of text (or images) in a high-dimensional space.

```
"king"    → [0.2, 0.8, 0.1, 0.9, ...]    384 dimensions
"queen"   → [0.2, 0.7, 0.1, 0.8, ...]    ← close to "king"
"banana"  → [0.9, 0.1, 0.7, 0.2, ...]    ← far from "king"
```

Key insight: Similar meanings → similar vectors → small distance

Embeddings are generated by ML models:

- ELSER (Elastic) — sparse embeddings, English
- EmbeddingGemma-300M (Google) — 768-dim dense
- text-embedding-3-small (OpenAI) — 1536-dim dense

dense_vector Field Type

```
PUT /vector-demo
{
  "mappings": {
    "properties": {
      "title": { "type": "text" },
      "overview": { "type": "text" },
      "embedding": {
        "type": "dense_vector",
        "dims": 3,
        "index": true,
        "similarity": "cosine"
      }
    }
  }
}
```

dense_vector Parameters

Parameter	Purpose
<code>dims</code>	Vector dimensions (3 for demo, 768-1536 in production)
<code>index</code>	Enable HNSW indexing for fast kNN search
<code>similarity</code>	Distance metric: <code>cosine</code> , <code>l2_norm</code> , <code>dot_product</code>

Similarity Metrics

Cosine Similarity

$$\frac{A \cdot B}{|A| \times |B|}$$

L2 Norm (Euclidean)

$$\sqrt{\sum(A_i - B_i)^2}$$

Dot Product

$$\sum(A_i \times B_i)$$

Metric	Best For	Note
cosine	Text embeddings	Direction matters, not magnitude
l2_norm	Image/spatial	Actual distance between points
dot_product	Pre-normalized vectors	Fastest computation

Default choice: Use `cosine` for text search.

HNSW: How kNN Search is Fast

HNSW (Hierarchical Navigable Small World) — an approximate nearest neighbor algorithm:

```
Layer 2:  [A] ---- [D]           ← Long-range connections
           |         |
Layer 1:  [A] - [B] - [D] - [F]   ← Medium connections
           |   |   |   |   |
Layer 0:  [A]-[B]-[C]-[D]-[E]-[F] ← All nodes, local connections
```

- Multi-layer graph structure
- Search starts at top layer (coarse), drills down (fine)
- Trade-off: accuracy vs speed (controlled by `num_candidates`)
- Not exact — "approximate" nearest neighbors (ANN)

kNN Query

```
GET /movies-embeddings/_search
{
  "knn": {
    "field": "overview_embedding",
    "query_vector": [0.02, -0.05, 0.08, ...],
    "k": 5,
    "num_candidates": 50
  },
  "_source": ["title", "overview", "genres"]
}
```

Parameter	Purpose
<code>k</code>	Number of nearest neighbors to return
<code>num_candidates</code>	Candidate pool size (higher = more accurate, slower)

Rule of thumb: `num_candidates` = 5-10x of `k`.

kNN with Filtering (Pre-filtering)

```
GET /movies-embeddings/_search
{
  "knn": {
    "field": "overview_embedding",
    "query_vector": [0.02, -0.05, 0.08, ...],
    "k": 5,
    "num_candidates": 50,
    "filter": {
      "term": { "genres": "Drama" }
    }
  }
}
```

- Filter is applied **before** kNN search (pre-filtering)
- Only matching documents are considered as candidates
- Filters use the same Query DSL as regular searches

Loading the Embeddings Dataset

Our movies dataset includes pre-computed 768-dimensional embeddings:

```
python data/load_data.py --dataset movies --with-embeddings
```

This creates `movies-embeddings` with ~5100 movies, each containing:

Field	Type	Description
<code>title</code>	text	Movie title
<code>overview</code>	text	Plot summary
<code>genres</code>	keyword	Genre array
<code>vote_average</code>	float	Rating 0-10
<code>overview_embedding</code>	dense_vector (768)	Pre-computed embedding

Practice 4A

Vector Search Basics

`day4-exercises.md` — Part A (5 tasks) | ~20 min

ELSER & Semantic Text

Zero-config semantic search

What is ELSER?

Elastic Learned Sparse EncodeR — Elastic's built-in NLP model.

Feature	Detail
Type	Sparse embedding model
Language	English
Output	Sparse vectors (weighted token expansions)
Runs on	ML nodes (in-cluster, no external API)
Cost	Free (included with Elastic license)

Sparse vs Dense:

- **Dense:** `[0.2, 0.8, 0.1, ...]` — 384 floats, every dimension used
- **Sparse:** `{"prison": 2.1, "escape": 1.8, "freedom": 1.5, ...}` — weighted tokens

Deploying ELSER

Via Kibana (Recommended)

1. **Machine Learning** → **Trained Models**
2. Find `.elser_model_2_linux-x86_64`
3. Click **Download** → **Deploy**
4. Wait for status: **started**

ELSER download is ~1.7GB. Ensure your ML node has sufficient memory.

Via API

See the **Inference Endpoints** slide for the API approach.

Inference Endpoints

The gateway between your data and ML models:

```
PUT /_inference/sparse_embedding/my-elser-endpoint
{
  "service": "elser",
  "service_settings": {
    "num_allocations": 1,
    "num_threads": 1
  }
}
```

Testing Inference Endpoints

```
POST /_inference/sparse_embedding/my-elser-endpoint
{
  "input": "Two men escape from prison"
}
```

Returns weighted token expansion:

```
{
  "sparse_embedding": [{
    "prison": 2.1, "escape": 1.8, "flee": 1.2, "convict": 0.9, ...
  }]
}
```

ELSER expands the input into **semantically related tokens** with relevance weights.

semantic_text Field Type

The simplest way to add semantic search — embeddings generated automatically:

```
PUT /movies-semantic
{
  "mappings": {
    "properties": {
      "title": { "type": "text" },
      "overview": { "type": "text" },
      "overview_semantic": {
        "type": "semantic_text",
        "inference_id": "my-elser-endpoint"
      },
      "genres": { "type": "keyword" },
      "vote_average": { "type": "float" }
    }
  }
}
```

semantic_text: How It Works

- Just index text → ELSER generates embeddings **automatically**
- No pre-computation pipeline needed
- Embeddings update when documents are updated

```
// Copy text to both fields when indexing
{
  "overview": "Two imprisoned men bond over years...",
  "overview_semantic": "Two imprisoned men bond over years..."
}
```

overview for keyword search, **overview_semantic** for semantic search.

Indexing with semantic_text

```
POST /movies-semantic/_bulk
{"index": {"_id": "1"}}
{"title": "Shawshank Redemption",
 "overview": "Two imprisoned men bond...",
 "overview_semantic": "Two imprisoned men bond...",
 "genres": ["Drama"], "vote_average": 8.7}
{"index": {"_id": "2"}}
{"title": "The Godfather",
 "overview": "An aging crime patriarch...",
 "overview_semantic": "An aging crime patriarch...",
 "genres": ["Crime", "Drama"], "vote_average": 8.7}
```

Copy the same text to both `overview` and `overview_semantic`.

Semantic Query

```
GET /movies-semantic/_search
{
  "query": {
    "semantic": {
      "field": "overview_semantic",
      "query": "movies about escaping from prison"
    }
  }
}
```

ELSER understands meaning:

- "escaping from prison" → finds "The Shawshank Redemption"
- Even though the overview says "imprisoned men" and "redemption" (not "escaping" or "prison")
- A keyword search for "escaping prison" would **miss** this document!

Keyword vs Semantic: Keyword Search

```
# Keyword search – matches exact tokens
GET /movies-semantic/_search
{
  "query": {
    "match": { "overview": "escaping prison" }
  }
}
```

Might **miss** Shawshank — no exact "escaping" or "prison" in overview.

Keyword vs Semantic: Semantic Search

```
# Semantic search – matches meaning
GET /movies-semantic/_search
{
  "query": {
    "semantic": {
      "field": "overview_semantic",
      "query": "escaping prison"
    }
  }
}
```

Finds Shawshank! "imprisoned" $\approx$ "prison", "redemption" $\approx$ "escaping"

When to Use semantic_text

Use semantic_text	Use dense_vector
Want simplicity	Need custom embedding model
English text	Multi-language support
ML nodes available	No ML infrastructure
Standard use cases	Sharing embeddings across systems
Auto-updated embeddings	Maximum indexing speed

Practice 4B

ELSER & Semantic Search

`day4-exercises.md` — Part B (4 tasks) | ~20 min

Hybrid Search with RRF

The best of both worlds

Why Hybrid?

Query	BM25 finds	kNN finds	Hybrid finds
"Shawshank"	Exact title match	Nothing (rare word)	Title match
"prison escape movie"	Docs with those words	Semantically similar	Both
"films about hope"	"hope" keyword matches	Thematic matches	Comprehensive

Neither method alone is perfect. Hybrid combines their strengths.

Reciprocal Rank Fusion (RRF)

Combines rankings from multiple retrievers without needing score normalization:

$$\text{RRF}(d) = \sum 1 / (k + \text{rank}_i(d))$$

Example:

Document	BM25 Rank	kNN Rank	RRF Score (k=60)
Movie A	1	5	$1/61 + 1/65 = 0.0318$
Movie B	3	1	$1/63 + 1/61 = 0.0323$
Movie C	2	10	$1/62 + 1/70 = 0.0304$

Movie B wins — ranked highly by **both** retrievers.

The Retriever API (ES 8.14+)

```
GET /movies-embeddings/_search
{
  "retriever": { "rrf": {
    "retrievers": [
      { "standard": { "query": {
        "match": { "overview": "space adventure" }
      } }},
      { "knn": {
        "field": "overview_embedding",
        "query_vector": [/* 768 dims */],
        "k": 10, "num_candidates": 50
      } }
    ],
    "rank_window_size": 100,
    "rank_constant": 60
  } }
}
```

RRF Parameters

Parameter	Default	Effect
<code>rank_constant</code>	60	Higher = more equal weighting across ranks
<code>rank_window_size</code>	100	How many results from each retriever to consider

`rank_constant` tuning

k=1: Top result gets ~50% of score weight (steep dropoff)
k=60: Balanced – good default for most cases
k=1000: All ranks get similar weight (nearly flat)

`rank_window_size` tuning

Small (50): Fast, but may miss relevant docs ranked 50–100
Large (200): Better recall, considers more candidates

Three-way Hybrid: BM25 + kNN + Semantic

```
GET /movies-semantic/_search
{
  "retriever": { "rrf": { "retrievers": [
    { "standard": { "query": { "multi_match": {
      "query": "prison escape",
      "fields": ["title^2", "overview"]
    } } } },
    { "knn": {
      "field": "overview_embedding",
      "query_vector": [/* embedding */],
      "k": 10, "num_candidates": 50
    },
    { "standard": { "query": { "semantic": {
      "field": "overview_semantic",
      "query": "escaping from prison"
    } } } }
  ] } }
}
```

Adding Filters to Hybrid Search

Filters must be applied to **each retriever** separately:

```
{
  "retriever": { "rrf": { "retrievers": [
    { "standard": { "query": { "bool": {
      "must": { "match": { "overview": "war" } } },
      "filter": { "range": { "vote_average": { "gte": 8 } } } }
    }
  ] } },
  { "knn": {
    "field": "overview_embedding",
    "query_vector": [/* ... */],
    "k": 10, "num_candidates": 50,
    "filter": { "range": { "vote_average": { "gte": 8 } } }
  } }
}
```

The same filter appears in **both** retrievers — there's no global filter in RRF.

Practice 4C

Hybrid Search with RRF

`day4-exercises.md` — Part C (5 tasks) | ~25 min

Advanced Topics

Production-ready semantic search

Quantization: Reducing Memory

int8 scalar quantization reduces vector memory by ~4x:

```
PUT /movies-quantized
{
  "mappings": {
    "properties": {
      "overview_embedding": {
        "type": "dense_vector",
        "dims": 768,
        "index": true,
        "similarity": "cosine",
        "index_options": {
          "type": "int8_hnsw"
        }
      }
    }
  }
}
```

Quantization: Trade-offs

	Float32 (default)	Int8 (quantized)
Memory per vector	$768 \times 4 = 3,072$ bytes	$768 \times 1 = 768$ bytes
Accuracy	Exact	~95-99% of original
Use when	< 1M documents	> 1M documents

4x memory reduction with minimal accuracy loss — essential for large-scale vector search.

Chunking Long Documents

Long documents need to be split into chunks for effective embedding:

```
Original: 5000-word article
```

```
↓
```

```
Chunk 1: words 1-500      → embedding_1
```

```
Chunk 2: words 450-950    → embedding_2  (50 word overlap)
```

```
Chunk 3: words 900-1400   → embedding_3
```

```
...
```

- Embedding models have a **token limit** (typically 256-512 tokens)
- Use **overlapping windows** to preserve context at boundaries

Chunking: Index Schema

```
PUT /articles-chunked
{
  "mappings": {
    "properties": {
      "article_id": { "type": "integer" },
      "chunk_id": { "type": "integer" },
      "chunk_text": { "type": "text" },
      "chunk_embedding": {
        "type": "dense_vector",
        "dims": 384,
        "index": true,
        "similarity": "cosine"
      }
    }
  }
}
```

Use `collapse` on `article_id` to deduplicate results.

Pre-filtering vs Post-filtering

Pre-filtering (recommended)

```
"knn": {  
  "field": "embedding",  
  "query_vector": [...],  
  "k": 10,  
  "num_candidates": 100,  
  "filter": { "term": { "genre": "Drama" } }  
}
```

Filter applied **BEFORE** kNN → always returns k results matching filter.

Post-filtering

```
"knn": { "field": "embedding", "query_vector": [...], "k": 10 },  
"post_filter": { "term": { "genre": "Drama" } }
```

Filter applied **AFTER** kNN → may return **fewer than k** results.

	Pre-filter	Post-filter
Result count	Always k	$\leq k$
Use when	Default choice	Need facet counts from unfiltered results

Custom Embedding Models: OpenAI

```
PUT /_inference/text_embedding/openai-embeddings
{
  "service": "openai",
  "service_settings": {
    "api_key": "sk-...",
    "model_id": "text-embedding-3-small"
  }
}
```

- 1536 dimensions, multi-language
- Requires API key + external network call

Custom Embedding Models: Hugging Face

```
PUT /_inference/text_embedding/hf-embeddings
{
  "service": "hugging_face",
  "service_settings": {
    "api_key": "hf_...",
    "url": "https://api-inference.huggingface.co/...all-MiniLM-L6-v2"
  }
}
```

External models: better quality, multi-language, but add **latency + cost** and data leaves your cluster.

Approximate kNN (default — HNSW)

```
GET /movies-embeddings/_search
{
  "knn": {
    "field": "overview_embedding",
    "query_vector": [...],
    "k": 10,
    "num_candidates": 100
  }
}
```

- Uses HNSW graph for fast approximate search
- Trade-off: accuracy vs speed (controlled by `num_candidates`)

Exact kNN (brute-force — script_score)

```
GET /movies-embeddings/_search
{
  "query": {
    "script_score": {
      "query": { "match_all": {} },
      "script": {
        "source": "cosineSimilarity(params.qv, 'overview_embedding') + 1.0",
        "params": { "qv": [/* 768 dims */] }
      }
    }
  }
}
```

Exact is **$O(n)$** — only viable for small datasets (<10K docs) or evaluation.

Production Architecture

Application → Generate embedding → Hybrid search (BM25 + kNN)



Embedding API
(OpenAI / HF / local)



Elasticsearch Cluster
Data nodes + ML nodes (ELSER)

Key decisions:

Decision	Options
Embedding source	ELSER (in-cluster) vs external API
kNN mode	Approximate (HNSW) vs exact (script_score)
Memory	Float32 vs int8 quantization
Long docs	Chunking with overlap + collapse

Practice 4D

Advanced Techniques (Bonus)

`day4-exercises.md` — Part D (4 tasks) | ~20 min

Course Wrap-up

4-Day Recap

Day	Topic	Key Skills
Day 1	Fundamentals	Cluster concepts, CRUD, inverted index
Day 2	Query DSL & ES QL	match, bool, highlighting, pipe syntax
Day 3	Indexing & Analysis	Bulk ops, analyzers, mappings, aggregations
Day 4	Semantic Search	Vectors, ELSER, hybrid RRF, production tips

What to Explore Next

- **Elastic Observability** — Logs, metrics, APM
- **Elastic Security** — SIEM, endpoint protection
- **Index Lifecycle Management** — Automated rollover/delete
- **Cross-cluster Search** — Federated search across clusters
- **Inference Pipelines** — Enrich documents on ingest
- **ES|QL** — Continues to gain features each release

Resources

- **This repository** — keep it for reference and continued practice
- [Elastic Documentation](#)
- [ES|QL Reference](#)
- [Vector Search Guide](#)
- [ELSER Documentation](#)
- [Elasticsearch Labs Blog](#)

Thank You!

Congratulations on completing the course!

4-Day Elasticsearch Course | Elasticsearch 8.19